

Algorithms

Lecture #21

Syed Hamed Raza

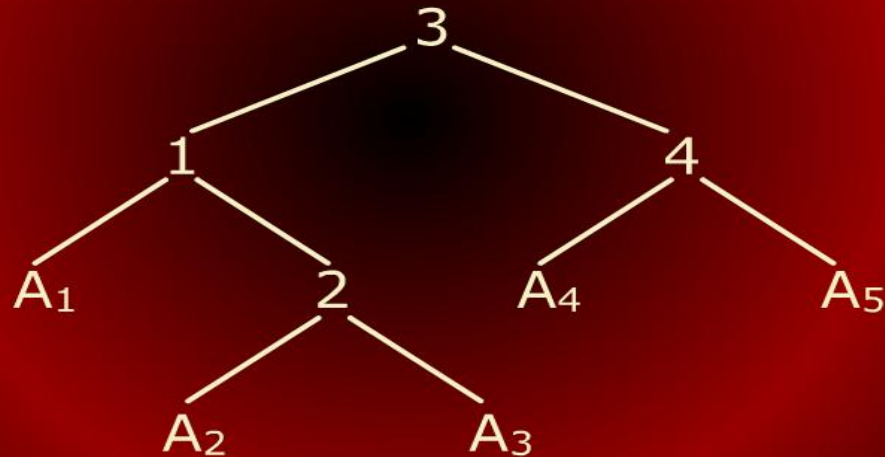
Design & Analysis of Algorithms

Chain Matrix Multiplication

Chain Matrix Multiplication

Optimal Order for multiplication

$((A_1(A_2A_3))(A_4A_5))$



Chain Matrix Multiply Algorithm

Chain Matrix Multiply Algorithm

MATRIX-CHAIN(p, N)

```
1  for  $i = 1, N$ 
2  do  $m[i, i] \leftarrow 0$ 
3  for  $L = 2, N$ 
4  do
5  for  $i = 1, n - L + 1$ 
6  do  $j \leftarrow i + L - 1$ 
7   $m[i, j] \leftarrow 1$ 
8  for  $k = 1, j - 1$ 
```

Chain Matrix Multiply Algorithm

Chain Matrix Multiply Algorithm

MATRIX-CHAIN(p, N)

```
6  do  $j \leftarrow i + L - 1$ 
7   $m[i, j] \leftarrow 1$ 
8  for  $k = 1, j - 1$ 
9  do  $t \leftarrow m[i, k] + m[k + 1, j]$ 
       $+ p_{i-1} \cdot p_k \cdot p_j$ 
10 if ( $t < m[i, j]$ )
11 then  $m[i, j] \leftarrow t; s[i, j] \leftarrow k$ 
```

Chain Matrix Multiply Algorithm

Chain Matrix Multiply Algorithm

Analysis

- There are three nested loops.
- Each loop executes a maximum n times.
- Total time is thus $\Theta(n^3)$.

Chain Matrix Multiply Algorithm

Chain Matrix Multiply Algorithm

Extracting the final sequence

MULTIPLY(i, j)

- 1** if ($i = j$)
- 2** then return $A[i]$
- 3** else $k \rightarrow s[i, j]$
- 4** $X \rightarrow \text{MULTIPLY}(i, k)$
- 5** $Y \rightarrow \text{MULTIPLY}(k + 1, j)$
- 6** return $X \cdot Y$

0/1 Knapsack Problem

0/1 Knapsack Problem

- Given a knapsack with maximum capacity W , and a set S consisting of n items
- Each item i has some weight w_i and value v_i (all w_i , v_i and W are integer values)
- Problem: How to pack the knapsack to achieve maximum total value of packed items?

0/1 Knapsack Problem

0/1 Knapsack Problem

Item i	Weight w_i	Value v_i
1	2	3
2	3	4
3	4	5
4	5	8
5	9	10



Knapsack can hold $W = 20$

0/1 Knapsack Problem

0/1 Knapsack Problem

- Mathematically, the problem is

$$\text{maximize } \sum_{i \in T} v_i$$

$$\text{subjected to } \sum_{i \in T} w_i \leq w$$

- The problem is called a "0-1" problem, because each item must be entirely accepted or rejected.

0/1 Knapsack Problem

0/1 Knapsack Problem

Try the brute-force solution

- Since there are n items, there are 2^n possible combinations of the items (an item either chosen or not).
- We go through all combinations and find the one with the most total value and with total weight less or equal to W
- Running time will be $O(2^n)$.

0/1 Knapsack Problem

0/1 Knapsack Problem

Can we do better?

- Yes, with an algorithm based on dynamic programming
- We need to carefully identify the subproblems

0/1 Knapsack Problem

0/1 Knapsack Problem

Let us try this:

- If items are labelled $1, 2, \dots, n$, then a subproblem would be to find an optimal solution for

$$S_k = \text{items labelled } 1, 2, \dots, k$$

0/1 Knapsack Problem

0/1 Knapsack Problem

This is a valid subproblem definition.

- **The question is:**
can we describe the final solution S_n in terms of subproblems S_k ?
- Unfortunately, we cannot do that.

Here is why.

0/1 Knapsack Problem

0/1 Knapsack Problem

Solution S₄

	Item i	Weight w_i	Value v_i
•Items chosen are 1, 2, 3, 4	1	2	3
	2	3	4
	3	4	5
•Total weight: $2+3+4+5 = 14$	4	5	8
	5	9	10

- Total value: $3 + 4 + 5 + 8 = 20$

0/1 Knapsack Problem

0/1 Knapsack Problem

Solution S_5

- Items chosen are
1, 3, 4, 5

- Total weight:
 $2+4+5+9 = 20$

- Total value: $3 + 5 + 8 + 10 + 26$
 S_4 is not part of S_5 !!

Item	w_i	v_i
1	2	3
2	3	4
3	4	5
4	5	8
5	9	10

0/1 Knapsack Problem

0/1 Knapsack Problem

- The solution for S_4 is not part of the solution for S_5 .
- So our definition of a subproblem is flawed and we need another one.
- Unfortunately, we cannot do that.

Here is why.

0/1 Knapsack Problem: DP Approach

0/1 Knapsack Problem: DP Approach

The Dynamic Programming Approach

- For each $i \leq n$ and each $w \leq W$, solve the knapsack problem for the first i objects when the capacity is w .
- Why will this work?
- Because solutions to larger subproblems can be built up easily from solutions to smaller ones.

0/1 Knapsack Problem: DP Approach

0/1 Knapsack Problem: DP Approach

- We construct a matrix
 $V[0 \dots n, 0 \dots W]$.
- For $1 \leq i \leq n$, and $0 \leq j \leq W$,
 $V[i, j]$ will store the maximum value
of any set of objects
 $\{1, 2, \dots, i\}$ that can fit into a
knapsack of weight j .

0/1 Knapsack Problem: DP Approach

0/1 Knapsack Problem: DP Approach

- $V[n, W]$ will contain the maximum value of all n objects that can fit into the entire knapsack of weight W .

0/1 Knapsack Problem

0/1 Knapsack Problem

- To compute entries of V we will imply an inductive approach.
- As a basis, $V[0, j] = 0$ for $0 \leq j \leq W$ since if we have no items then we have no value.
- We consider two cases:

O/1 Knapsack

Design and Analysis of Algorithms